

---

```
print('My variable is', myVariable)
```

---

However, if you try to use `print()` to help you debug the contents of an object, the results are not particularly helpful. For example, here we create a `Vector` object and print it:

---

```
oVector = Vector(3, 4)
print('My vector is', oVector)
```

---

This is what is printed:

---

```
<Vector object at 0x10361b518>
```

---

This tells us that we have an object instantiated from the `Vector` class and shows the memory address of that object. However, in most cases, what we really want to know are the values of the instance variables in the object at that moment. Luckily, we can use magic methods for that.

There are two magic methods that can be useful in getting information (as strings) from an object:

- The `__str__()` method is used to create a string representation of an object that can be read easily by humans. If client code makes a call to the `str()` built-in function and passes in an object, Python will call the magic method `__str__()` if it is present in that class.
- The `__repr__()` method is used to create an unambiguous, possibly machine-readable string representation of the object. If client code makes a call to the `repr()` built-in function and passes in an object, Python will attempt to call the magic method `__repr__()` in that class, if present.

I'll show the `__str__()` method, as it's more generally used for simple debugging. When you call the `print()` function, Python calls the built-in `str()` function to convert each argument into a string. For any argument that does not have a `__str__()` method, this function formats a string that contains the type of the object, the words "object at," and the memory address, then returns the resulting string. That's why we see the earlier output containing the memory address.